Getting started with Spark & batch processing frameworks

Hoa Nguyen
May 21, 2020 · 10 min read

> *What you need to know before diving into big data processing with Apache Spark and other frameworks.*

When I was an Insight Data Engineering Fellow in 2016, I knew very little about Apache Spark prior to starting the program. Worse, documentation seemed sparse and wasn't helpful to beginners.

Four years later, Spark is now much more prevalent and there are numerous online guides that can get you started in a matter of minutes, but I still find a few misconceptions that are worth dispelling for the beginner (Note: I'm primarily talking about batch processing available under Spark, although many of the concepts still apply if you were to use Spark Streaming or Spark Structured Streaming framework).

Spark is not always the right tool to use

Spark is not magic, and using it will not automatically speed up data processing. In fact, in many cases, adding Spark will slow your processing, not to mention eat up a lot of resources. That is because there's a lot of overhead in running Spark. It's a distributed computing framework, which means that it was designed to run on multiple machines configured to talk to one another in a master-worker configuration.

A master-worker configuration means the master acts as a driver or controller, and when a Spark "action" is invoked, the Spark engine will use a "Directed Acyclic Graph" that was created via previous Spark "transformations" to farm out the processing to workers in the Spark cluster.

This is a simplified explanation of how Spark works but it's worth noting that if you were to code something like Spark from scratch, you'd have to write remote procedure calls and manage threads in your program. You'd have to build all of this infrastructure, but you could call whatever processing you want, whenever you want and however you want it — all potentially customized to fit your needs.

But if you do use Spark, that infrastructure, including additional features, such as fault tolerance, is already built for you. The tradeoff is that you'd have to follow Spark's programming model and you wouldn't have the same amount of control as if you built the infrastructure yourself. It's a small amount of control to give up for what you gain — but you also can't think of using Spark the same way as if you were to write your own code from scratch.

If Spark isn't always the first right tool to use, what is? For the majority of many companies working in the data space, Python code or even just a well-designed database would suffice. It really depends on what you need to do and how much data you must process. For instance, aggregating 10 gigabytes of data could likely be done with just Python on a single machine without having to involve Spark.

There is no set formula for when using a distributed computing framework, such as Spark, would be the "right" answer. Typically, you would determine what processing logic is needed and how much time and resources would be required using just Python, or your programming language of choice, without the use of distributed computing frameworks. Then you could weigh that against the benefits and drawbacks (e.g., more overhead, more complicated set-up) that come with adding something such as Spark.

Sometimes, developers justify the use of Spark because they anticipate their needs to grow over time and want to have the architecture in place to scale. However, that can be a faulty premise because it can be hard to predict the direction of future growth (e.g., will you get more data, different kinds of data, or both, or will what you query from your data change?). Again, using Spark doesn't guarantee scalability. If you switch to Spark when all you needed was Python, you may end up having to unnecessarily maintain Spark code and infrastructure and then later determine it's not the bottleneck or architectural component that you needed to scale.

Getting started with Spark

In perusing much of the freely available published material on getting started with Spark, many recommend getting started with a local installation (on one machine) and then moving to having it installed on a cluster of machines. My advice: skip the one-machine Spark installation and move directly to a cluster installation. It's a bit more involved, but installing Spark on one machine is already fairly involved, and so once you install on one, it's not that big of a jump to install it on a cluster of machines.

A major disadvantage of installing, and using Spark, on one machine is that it lulls you into thinking you've conquered Spark. Everything works as promised, especially when you process a small amount of data, and you come out thinking, "Gee, that was easy." It's only when you go beyond a local installation and process a large amount of data in a distributed manner on Spark — the way it was designed to work — that you'll run into issues, potentially with capacity, memory, race conditions, and other obstacles. It's only through these struggles, that you'll learn the most about Spark.

My Insight colleague, Sriram Baskaran, wrote a fantastic blog on how to install Spark in a cluster mode with step-by-step instructions. His instructions assume that you have a cluster of machines available to install Spark. If you want help with how to provision your machines on AWS, here's another walk-through to help you do that first. If you follow those instructions to the end, you can skip the "Step 0.5: Setting up Keyless SSH" section on Sriram's blog.

Once you've successfully installed Spark on to your cluster, be sure to check out the open-source "Quick Start" guide for a basic walk through of the interactive Spark shell (`spark-shell`)— which is a great way to learn about Spark commands and test some of its functionality interactively. If you don't know Scala or Java, I'd highly recommend staying with the Python API and Pyspark.

Keep in mind that `spark-shell` is meant to be an interactive sandbox. To run a Spark job that stands on its own, you'll want to write a self-contained application, and then pass that code to your Spark cluster using the command, `spark-submit`. In order to run your code using the distributed Spark cluster and not on your local machine, be sure and add the `--master` flag to your 'spark-submit' job.

If you are using Python (again, stick with Python unless you know Scala or Java or absolutely need APIs that are not available with Python), you'll want to learn about Spark's DataFrame API, which can be used interchangeably with Spark SQL calls.

Keep in mind the Spark DataFrame API and Spark SQL calls use an underlying execution engine that includes optimizations not available if you were to use Spark's native Resilient Distributed Datasets (RDDs). RDDs are still at the center of Spark's engine, and they're an important concept to understand and are heavily used under the covers by the DataFrame API and Spark SQL calls. To put it another way, when you make a Spark SQL call, the underlying API is invoking core Spark functions that rely on the concept of RDDs but all of that work is abstracted from the user, in addition to other optimizations the Spark engine throws in with the DataFrame API and other Spark SQL calls.

Those optimizations are a big reason why you'd want to use this newer API. If you stumble on code or documentation that specify RDD transformations and actions — the older but core API — it's highly likely the documentation or code may be a bit dated.

To use Hadoop or not

Using the Spark engine does not require installing Apache Hadoop or making use of the Hadoop Distributed File System but you'll find that having access to HDFS on your cluster can be advantageous. Here are detailed instructions on how to install and test

an installation of Hadoop on AWS. One of the prerequisites to installing Hadoop is to provision a cluster of machines. If you need instructions on how to do that, they can be found here.

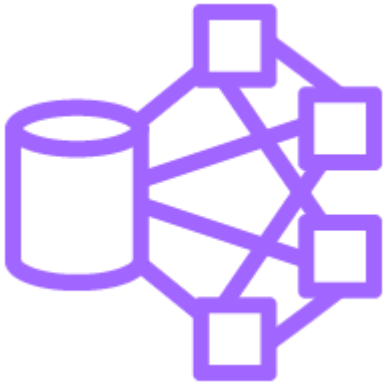
Other tools

Earlier, I said that Spark is not always the right tool to use, and it's also not the only tool that can handle a large amount of data. Below are a few other tools that can be used in building data pipelines:

Presto: An open-source distributed SQL query engine that can be used to query large amounts of data where it is stored, including HDFS, Amazon S3 and a number of other sources. Presto, which was open-sourced in 2013 and predates the rise of Spark SQL, became popular due to its reliance on the SQL standard to query large amounts of data. Originating out of Facebook, Presto is designed for data warehousing and analytics use cases. It relies on a coordinator-worker architecture that was initially designed to work with HDFS but has been extended to connect to a number of data stores, including relational databases and NoSQL databases, such as Apache Cassandra. Companies, such as Pinterest and Lyft, use Presto.

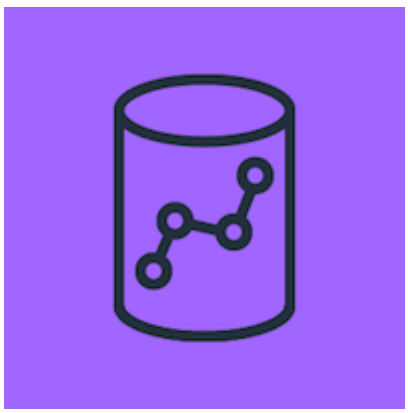
Amazon Athena: If you're intrigued by Presto but want to use it as a managed service to query data in S3 buckets using SQL, Amazon Athena has got you covered. You can think of Athena as an installation of Presto that Amazon manages for you and then charges you on a per query basis. Many companies use Athena so the skill that you pick up will be useful. Some challenges you might want to consider are how to optimize

your queries and keep costs down (Athena and Amazon Glue, which is often used in conjunction with it, can be very expensive and you can blow though your budget very quickly). Companies, such as Atlassian, use Athena.



AWS EMR

Amazon EMR: Managed services have grown in popularity in recent years, particularly because they allow you to avoid installation and management headaches and are available when you need them. Say you're interested in running Spark and Presto jobs but you're not interested in installing and maintaining the cluster of machines that would be required — enter Amazon's EMR service. Using EMR will eliminate the need to manually provision ec2 instances, download and install Spark or Presto on those instances and keep them running — Amazon will do those tasks for you and then charge you an additional premium cost per instance and hour. But while you save on some installation tasks, you still must configure EMR, including type of instances and auto scaling features, that would require engineering considerations. Companies, such as Yelp and Zillow, uses Amazon EMR.



Redshift

snowflake

Google BigQuery

Amazon Redshift / Google Big Query / Snowflake: Spark and many of the frameworks discussed earlier are focused on processing data and don't really deal with how to make that information available to analysts and others for consumption. If providing a data warehouse or analytics platform where users can query the data according to their specific needs and use cases, you might consider a data warehouse through a managed service, such as Amazon Redshift or, if you're on Google, BigQuery. Snowflake, which provides its data warehouse solution on both Amazon and Google, as well as others, also has become highly popular in recent years for its ease of use and scalability. Keep in mind that just the mere act of creating a cluster (e.g., simply setting it up) will incur a charge so this is definitely a place where you can bulldoze through your budget. Companies, such as Equinox and Intuit use Redshift while others, such as New York Times and Spotify use Big Query. Capital One is a Snowflake customer.

Amazon Redshift Spectrum: Amazon Redshift Spectrum is different from the other tools discussed, but also similar in some ways. You can think of it as similar to Amazon Athena in that it queries data sitting in S3 buckets but, instead of using Presto, it uses an Amazon Redshift cluster. That Redshift cluster is only used to query the data in S3 bucket but not to store the data. Remember that the mere act of creating a cluster means you'll be paying for every hour the cluster is set up and in existence, regardless of whether you are firing any queries. Again, the main takeaway is that it can be an expensive option if you're not careful. Companies, such as iHeartRadio, uses Spectrum.

Apache Airflow: If you've written a job to transform and process data, whether it's code written in Python or a Spark job, chances are you'll want to run that job again with another dataset — for instance, as new data arrives or is updated. Instead of manually running that job again sometime in the future, it'd be great to schedule that job. There might be other code or script that you'd want to run if that job succeeded, or if it failed. This is where a workflow scheduler, such as Airflow would come in handy. Airflow is a tool that was open sourced by Airbnb and is now in use by many companies, including Pandora.

Apache NiFi: If you would like to manage data flows and control how they are passed between systems, NiFi could be a good option. NiFi can run in a cluster mode with the help of Apache Zookeeper to elect a "cluster coordinator." NiFi also comes with a graphical user interface that allows you to visualize dataflows and make any needed changes. Companies, such as Looker, use NiFi.

. . .

Interested in seeing more content like this? *Sign up for our newsletter, and get updates on the latest resources and upcoming events.*

Thanks to Bastian Haase and Ashwin Kumar.

Insight Data Engineering | Apache Spark | Apache Airflow | Python | API

Medium

About Help Legal

Get the Medium app

Download on the App Store

GET IT ON Google Play